

Compléments d'Algorithmique

Chapitre 2 : Structures de données

Hadrien Mélot
Service d'Algorithmique
Département d'Informatique
Faculté des Sciences



Chapitre 2. Structures de données

- 1 Qu'est-ce qu'une structure de données ?
- 2 Notions utiles préalables
- 3 Tuples
- 4 Listes
- 5 Dictionnaires
- 6 Ensembles
- 7 Complexités
- 8 Quand utiliser quelle structure de données ?
- 9 Remarques et opérations fréquentes

Qu'est-ce qu'une structure de données ?

Qu'est-ce qu'une structure de données ?

Conceptuellement, une **structure de données** est un moyen de **stocker** et de **manipuler** des données de manière à ce qu'elles puissent être utilisées efficacement.

En pratique, une structure de données est une implémentation d'un tel concept, et les langages de haut niveau offrent des structures de données prédéfinies.

Par exemple, en Python, vous en connaissez déjà : les chaînes de caractères, les tuples et les listes.

Qu'est-ce qu'une structure de données ?

Une structure de données peut être

- **statique** : la taille de la structure est fixée à la création ; ou
- **dynamique** : la taille de la structure peut varier au cours du temps.

Egalement, une structure de données peut être

- **homogène** : tous les éléments de la structure sont du même type ; ou
- **hétérogène** : les éléments peuvent être de types différents.

Enfin, une structure de données peut être

- **mutable** ou **immuable** ;
- **ordonnée** ou **non ordonnée**.

Exemples

- une chaîne en Python est une structure de données statique, immuable, ordonnée et homogène ;
- un tuple en Python est une structure de données statique, immuable, ordonnée et hétérogène ;
- une liste en Python est une structure de données dynamique, mutable, ordonnée et hétérogène ;
- un tableau en C est une structure de données statique, mutable, ordonnée et homogène (voir chapitre ultérieur) ;
- un ArrayList en Java est une structure de données dynamique, mutable, ordonnée et homogène (voir cours *Algorithmique et programmation orientée objets*).

Nous verrons deux structures non-ordonnées : les dictionnaires et les ensembles. En Python, les structures statiques sont immuables, ce qui n'est pas le cas dans tous les langages.

Aperçu du chapitre

Dans ce chapitre nous allons

- (re)voir les structures de données de base en Python :
 - ▶ tuples ;
 - ▶ listes ;
 - ▶ dictionnaires ;
 - ▶ ensembles.
- découvrir des notions liées à la manipulation de ces structures :
 - ▶ fonction avec un nombre variable d'arguments ;
 - ▶ hachage ;
 - ▶ fonctions lambda ;
 - ▶ opérations `map`, `reduce` et `filter`.

Pour en savoir plus

Dans un chapitre ultérieur de ce cours, nous verrons un moyen pour créer ses **propres structures de données dynamiques**.

Vous en verrez également des plus avancées dans d'autres cours

- Algorithmique et programmation orientée objets (B1)
- Structures de données I (B2)
- Structures de données II (B3)

Notions utiles préalables

Utilisation de f-strings pour formater les chaînes en Python

Cela n'avait pas été présenté dans le cours d'*Algorithmique et programmation impérative*, mais les f-strings sont très utiles pour formater les chaînes en Python.

- La syntaxe `f"texte {variable} texte"` permet d'insérer directement la valeur d'une variable dans une chaîne de caractères.

- Exemple :

```
>>> x = 42
>>> print(f"Ma variable vaut {x}")
Ma variable vaut 42
```

- Les expressions à l'intérieur des crochets peuvent être n'importe quel calcul ou appel de fonction.

Hachage

La fonction `hash()` en Python permet de générer un entier unique (`hash`) pour un objet `immuable`.

Remarque : Les objets mutables (ex : listes, dictionnaires) ne sont pas hachables.

Cette fonction est notamment utilisée par Python pour opérer sur certaines structures de données de manière efficace.

Hachage

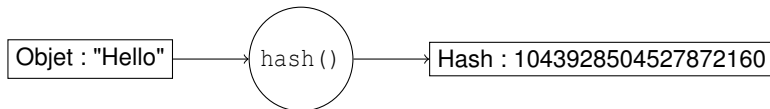
Exemple 1 : Hash d'un entier et d'une chaîne

```
>>> print(hash(42))  
42  
>>> print(hash("Hello"))  
1043928504527872160  
>>> print(hash((1, 2, 3)))  
529344067295497451
```

Exemple 2 : Objet non hachable

```
>>> print(hash([1, 2, 3]))  
TypeError: unhashable type: 'list'
```

Hachage



Interprétation :

- Chaque objet immuable reçoit un identifiant (*hash*) qui est généralement unique, mais des “collisions” sont possibles¹.
- Ce *hash* est notamment utilisé (de manière automatique) par Python pour un accès rapide dans les dictionnaires et les ensembles (voir plus loin).

1. Voir cours de Structure de Données II

Éléments communs aux Structures de données

Les structures de données que nous allons voir partagent plusieurs éléments communs. Nous pourrions donc réutiliser des éléments que nous connaissons déjà !

- Elles ont une **longueur** : on peut connaître le nombre d'éléments qu'elles contiennent (fonction `len()`).
- On peut savoir si un élément est présent dans la structure grâce à l'opérateur `in`.

```
>>> print(len([1, 2, 3]))
3
>>> print(42 in (1, 2, 3, 42))
True
```

Éléments communs aux Structures de données

- Elles sont **itérables** via une boucle `for`.
- On peut trier les éléments de la structure si ceux-ci sont **comparables** (voir ci-après) avec la fonction `sorted()`.
- D'autres opérateurs (comme la concaténation `+` ou le slice `[ : ]`) s'appliquent à ces structures quand cela à du sens.

```
>>> for c in "Hello":
...     print(c, end='')
H*e*1*1*o*
>>> print(sorted("Bonjour"))
['B', 'j', 'n', 'o', 'o', 'r', 'u']
>>> print((1, 2) + (3, 4))
(1, 2, 3, 4)
>>> print([1, 2, 3, 4][1:3])
[2, 3]
```

Qu'est-ce qui distingue les structures de données ?

Ce qui va différencier ces structures :

- la manière dont les données sont **stockées** et **organisées** ;
- les **opérations** qu'elles supportent ;
- la **complexité** de ces opérations ;
- les **propriétés** de chaque structure (ex : immuabilité, données hétérogènes ou non, etc.).

Nous allons apprendre à choisir la structure de données la plus adaptée à un problème donné.

Éléments comparables

Pour que les éléments d'une structure soient **comparables** entre eux, il faut pouvoir leur appliquer l'opérateur de comparaison `<`.

C'est le cas pour les types de base en Python (entiers, flottants, chaînes de caractères). Les tuples et les listes le sont aussi si les données sont homogènes et comparables entre elles, grâce à l'ordre lexicographique (voir ci après).

Éléments comparables

La fonction `sorted()` retourne une liste triée contenant les éléments comparables d'une structure de données.

Elle a deux arguments optionnels :

- `key` : le nom d'une fonction ou d'une méthode qui sera appliquée à chaque élément avant la comparaison ;
- `reverse` : un booléen qui indique si le tri doit être effectué en ordre décroissant (`False` par défaut).

```
>>> t = (3, 1, 4, 1, 5, 9)
>>> print(sorted(t))
[1, 1, 3, 4, 5, 9]
>>> print(sorted([3, 1, 4, 1, 5, 9], reverse=True))
[9, 5, 4, 3, 1, 1]
>>> print(sorted("Hello", key=str.lower))
['e', 'H', 'l', 'l', 'o']
```

Éléments comparables

Exemple : trier une liste de nombres en fonction du reste de la division par 3.

```
>>> def get_mod3(x):  
...     return x % 3  
>>> numbers = [5, 12, 3, 8, 1]  
>>> sorted_numbers = sorted(numbers, key=get_mod3)  
>>> print(sorted_numbers)  
[12, 3, 1, 5, 8]
```

Rendre des éléments comparables

Si les éléments d'une structure ne sont pas (toujours) comparables entre eux, on peut définir via `key` comment il faut les comparer.

```
>>> t = [1, 56, "hello"]
>>> sorted(t)
TypeError: '<' not supported between instances of 'str' and 'int'
>>> def my_key(x):
...     if isinstance(x, str):
...         return len(x)
...     else:
...         return x
...
>>> sorted(t, key=my_key)
[1, 'hello', 56]
```

Fonctions lambda

Une **fonction lambda** est une fonction anonyme, c'est-à-dire une fonction sans nom. Elle ne peut contenir qu'une seule expression.

Dans les exemples précédents, les fonctions `get_mod3` et `my_key` sont simples et ne sont utilisées qu'une seule fois. On peut les remplacer par des fonctions lambda.

Une fonction lambda est définie avec le mot-clef `lambda`, suivi des arguments, de `:` et de l'expression à retourner.

Syntaxe : `lambda arguments : expression`

```
>>> print(sorted([5, 12, 3, 8, 1], key=lambda x: x % 3))  
[12, 3, 1, 5, 8]
```

Fonctions lambda

Caractéristiques des fonctions lambda :

- **anonymes** : elles n'ont pas de nom ;
- **courtes** : elles ne peuvent contenir qu'une seule expression. Pour rappel, une expression produit une **valeur**. Il n'y a pas besoin de `return` : la valeur de l'expression est automatiquement retournée ;
- **utilisées localement** : elles sont souvent utilisées pour des fonctions simples qui ne sont utilisées qu'une seule fois ;
- **expressives** : elles permettent de rendre le code plus lisible.

Fonctions lambda

Si le code de la fonction que l'on veut transformer en fonction lambda contient une instruction `if-else` dont chacune des deux branches retourne une valeur, on peut la transformer en une expression en utilisant un `if` ternaire.

Syntaxe : `val_si_vrai if condition else val_si_faux`

```
>>> t = [1, 56, "hello"]
>>> sorted(t, key=lambda x: len(x) if isinstance(x, str) else x)
[1, 'hello', 56]
```

Contrairement à une **instruction** `if` classique, un `if` ternaire est une **expression** (et donc associée à une valeur), ce qui permet de l'utiliser dans une fonction lambda.

Fonctions lambda

Si une fonction contient une boucle `for`, on ne peut pas la transformer en une fonction lambda car `for` est une **instruction** et pas une expression.

Cependant, nous verrons le concept de **compréhension** dans ce chapitre. Une compréhension permet de créer une liste (ou d'autres structures) en une seule expression et sa syntaxe utilise un `for`.

Cet usage est limité à la création de certaines structures comme les listes, les ensembles ou les dictionnaires. Cela ne permet pas de transformer n'importe quelle boucle `for` en une seule expression.

Tuples

Rappels sur les tuples

Un **tuple** est une structure statique, ordonnée, hétérogène et immuable.

On définit un tuple en spécifiant les valeurs, séparées par des virgules.

```
>>> t = 'a', 'b'
>>> type(t)
<class 'tuple'>
>>> t = ('a', 'b', 'c')
>>> type(t)
<class 'tuple'>
>>> print(t)
('a', 'b', 'c')
```

L'utilisation des parenthèses est facultative, mais est souvent utilisée pour plus de clarté.

Rappels sur les tuples

Les valeurs peuvent être de n'importe quel type (données hétérogènes).

```
>>> data = ('a', 1, [1, 2])
```

Cependant, en pratique, les tuples sont souvent utilisés pour regrouper des valeurs de types identiques (données homogènes).

```
>>> words = ('Hello', 'World')
```

Rappels sur les tuples

Les valeurs sont indicées par des entiers (allant de 0 à $n - 1$, où en parcourant les indices à l'envers, allant de -1 à $-n$);

```
>>> t = ('a', 'b', 'c')
>>> print(t[0])
a
>>> print(t[-1])
c
```

Les tuples sont immuables.

```
>>> t[0] = 'z'
TypeError: 'tuple' object does not support item assignment
```

Un tuple avec une seule valeur

Pour distinguer syntaxiquement une unique valeur et un tuple contenant une seule valeur, on doit utiliser une virgule après la seule valeur du tuple.

```
>>> t = ('a',)
>>> print(type(t), t)
<class 'tuple'> ('a',)
>>> t = 'a',
>>> print(type(t), t)
<class 'tuple'> ('a',)
>>> t = ('a')
>>> print(type(t), t)
<class 'str'> a
```

Ce sont donc bien les virgules (plutôt que les parenthèses) qui définissent syntaxiquement les tuples. Celles-ci sont utilisées par convention (car proche de la notation mathématique des n -uples).

Assignment de tuples

Un tuple de **variables** peut se placer à gauche d'une assignation. Il faut que le membre droit soit un tuple d'expressions ayant le même nombre d'éléments.

```
>>> a = 17
>>> b = 21
>>> (a, b) = (b, a)
>>> print(a, b)
21 17
>>> a, b = 1 * 2, 3 + 4
>>> print(a, b)
2 7
>>> a, b = 1, 2, 3
ValueError: too many values to unpack (expected 2)
```

Assignment de tuples

De manière plus générale, le membre droit de l'assignation peut être une autre séquence, du moment que le nombre d'éléments correspond.

```
>>> a, b, c = 'xyz'
>>> print(a)
x
>>> uname, domain = 'jack@umons.ac.be'.split('@')
>>> print(uname)
jack
>>> print(domain)
umons.ac.be
```

Assigination de tuples

Strictelement parlant, une fonction ne peut retourner qu'une seule valeur. Utiliser un tuple comme valeur de retour permet de contourner cette limitation.

```
def min_max(t):  
    min = max = t[0]  
    for k in t:  
        if k > max:  
            max = k  
        if k < min:  
            min = k  
    return min, max  
  
>>> print(min_max(range(3,8)))  
(3, 7)
```

Opérateur scatter *

L'opérateur * (scatter) appliqué sur un tuple lors de l'appel à une fonction permet de séparer un tuple en une séquence d'arguments.

```
>>> help(divmod)
```

```
Help on built-in function divmod in module __builtin__:
```

```
divmod(...)
```

```
    divmod(x, y) -> (div, mod)
```

```
    Return the tuple ((x-x%y)/y, x%y).  Invariant: div*y + mod == x.
```

```
>>> t = (7,3)
```

```
>>> divmod(t)
```

```
TypeError: divmod expected 2 arguments, got 1
```

```
>>> divmod(*t)
```

```
(2, 1)
```

Comparaison de tuples

Si les données sont homogènes, les tuples sont comparables. Ils sont comparés élément par élément, de gauche à droite, et la comparaison s'arrête dès qu'une différence est trouvée. Si le nombre d'éléments est différent (mais les premières valeurs identiques), la séquence la plus courte est considérée comme étant la plus petite.

On parle d'ordre **lexicographique** (c'est le même comportement pour des chaînes de caractères).

```
>>> (1, 2, 3) < (1, 2, 4)
True
>>> (1, 2, 3) >= (1, 2, 3, 4)
False
>>> (1, 2, 3) > (1, 2)
True
>>> (1, 2, 3) == (1, 2, 3)
True
```

Exercice

Une développeuse de jeux vidéo veut stocker les scores des joueurs dans une structure immuable pour éviter toute modification accidentelle. Elle décide d'utiliser des tuples pour cela.

Notre mission :

- Créer un tuple `scores` contenant les scores de cinq joueurs (exemple : (1200, 850, 940, 1020, 760)).
- Afficher le score du troisième joueur.
- Trouver et afficher le score maximum et minimum.
- Trier les scores du plus élevé au plus bas et affichez-les.
- Un nouveau score de 990 doit être ajouté. Comme un tuple est immuable, comment "ajouter" ce score au tuple des scores ?

Listes

Rappels sur les listes

Une liste est une structure de données dynamique, ordonnée, hétérogène et mutable.

Une liste est définie par des crochets [] contenant des éléments séparés par des virgules. Si les données sont homogènes, les listes sont comparables grâce à l'ordre lexicographique.

```
>>> data = ['text', 2.0, 1, [2, 3]]
>>> len(data)
4
>>> print(data[0])
text
>>> print(data[3])
[2, 3]
>>> data[0] = 42
>>> data[-1][0] = 99
>>> print(data)
[42, 2.0, 1, [99, 3]]
```

Rappel des méthodes les plus courantes sur les listes

- La méthode `append` ajoute un nouvel élément à la fin de la liste ;
- La méthode `pop` supprime et retourne le dernier élément de la liste (ou l'élément se trouvant à un indice donné) ;
- La méthode `sort` trie les éléments de la liste (*in place*).

```
>>> t = ['z', 'b', 'c']
>>> t.append('d')
>>> print(t)
['z', 'b', 'c', 'd']
>>> deleted = t.pop()
>>> print(deleted)
d
>>> t.pop(1)
'b'
>>> print(t)
['z', 'c']
>>> t.sort()
>>> print(t)
['c', 'z']
```

D'autres méthodes utiles sur les listes

- La méthode `extend` prend une liste en argument et ajoute tous ses éléments en fin de la liste ;
- La méthode `count` compte toutes les occurrences d'une valeur passée en argument ;
- La méthode `index` retourne l'indice de la première occurrence d'une valeur donnée.

```
>>> t1 = ['a', 'b', 'c']
>>> t2 = ['a', 'e']
>>> t1.extend(t2)
>>> print(t1)
['a', 'b', 'c', 'a', 'e']
>>> t1.count('a')
2
>>> t.index('a')
0
```

D'autres méthodes utiles sur les listes

- La méthode `insert` permet d'insérer un élément à un endroit donné de la liste.
- La méthode `remove` supprime la première occurrence d'une valeur donnée.

```
>>> t = ['a', 'b', 'c']
>>> t.insert(0, 'z')
>>> print(t)
['z', 'a', 'b', 'c']
>>> t.insert(2, 'x')
>>> print(t)
['z', 'a', 'x', 'b', 'c']
>>> t.remove('b')
>>> print(t)
['z', 'a', 'x', 'c']
```

Comme on le rappellera, d'un point de vue complexité, il est plus efficace d'ajouter ou de supprimer des éléments en fin de liste avec `append` et `pop`.

Les méthodes qui ne renvoient pas de valeur

Certaines méthodes modifient la liste mais ne renvoient pas de valeur (plus précisément, renvoient `None`)

C'est le cas des méthodes `append`, `extend`, `insert`, `remove` et `sort`.

Ne l'oubliez pas, au risque de perdre les données.

```
>>> t = [3, 1, 42]
>>> print(t.append(9))
None
>>> print(t.remove(1))
None
>>> print(t)
[3, 42, 9]
>>> t = t.sort()
>>> print(t)
None
```

Exercice

Vous êtes en train de préparer une liste de courses, mais comme toujours, il y a des oublis et des changements de dernière minute !

Notre mission :

- Créer une liste `courses` contenant les éléments suivants : "pâtes", "tomates", "lait", "œufs", "fromage".
- Ajouter "pain" et "chocolat" à la liste.
- Supprimer "lait" de la liste (vous êtes intolérant·e au lactose).
- Remplacer "tomates" par "sauce tomate" (erreur de frappe).
- Afficher la liste finale triée par ordre alphabétique.

Dictionnaires

Dictionnaires

Un **dictionnaire**, comme une liste, est une structure de données dynamique, hétérogène et mutable, mais **non ordonnée**.

De plus, un dictionnaire est plus général qu'une liste dans le sens où :

- dans une liste, les indices doivent être des entiers ;
- dans un dictionnaire, ils peuvent être de n'importe quel type **hachable**.

Un dictionnaire peut être vu comme une correspondance entre un ensemble d'indices (les **clefs**) et de valeurs.

- à chaque clef correspond une valeur ;
- l'association entre une clef et une valeur est une **paire clef-valeur** ou un **élément** de la séquence.

Dictionnaires

Syntaxe :

- lors de sa création, on utilise des accolades { } ;
- pour associer des clefs aux valeurs, on écrit une séquence `clef : valeur` séparés par des virgules ;
- on peut également utiliser l'opérateur d'accès pour modifier un dictionnaire `d[clef] = valeur`.

Exemple.

Construisons un dictionnaire anglais-français : les clefs et les valeurs seront des chaînes de caractères.

```
>>> eng2fr = {}
>>> type(eng2fr)
<class 'dict'>
>>> eng2fr = {'one': 'un', 'two': 'deux', 'three': 'trois'}
>>> eng2fr['four'] = 'quatre'
>>> print(eng2fr)
{'one': 'un', 'two': 'deux', 'three': 'trois', 'four': 'quatre'}
```

Dictionnaires

On accède a un élément d'un dictionnaire en utilisant la clef entre crochets.

```
>>> print(eng2fr['two'])  
deux  
>>> print(eng2fr['five'])  
KeyError: 'five'
```

Dictionnaires

Depuis Python 3.7, l'ordre des valeurs dans un dictionnaire est celui dans lequel elles ont été insérées.

```
>>> eng2fr = {}
>>> eng2fr['two'] = 'deux'
>>> eng2fr['one'] = 'un'
>>> eng2fr['four'] = 'quatre'
>>> eng2fr['three'] = 'trois'
>>> print(eng2fr)
{'two': 'deux', 'one': 'un', 'four': 'quatre', 'three': 'trois'}
```

Cela n'est pas garanti dans les versions antérieures de Python.

Mais conceptuellement, l'ordre des éléments d'un dictionnaire **n'a pas vraiment d'importance**, car on accède aux éléments par leur clef. C'est la raison pour laquelle on considère qu'un dictionnaire est une structure **non ordonnée**.

Dictionnaires

Remarques :

- les valeurs peuvent être de n'importe quel type (comme pour les listes) ;
- les clefs doivent être **hachables** (et donc d'un type immuable comme les chaînes de caractères, les entiers ou les tuple) ;
- les clefs doivent être uniques : si on assigne une valeur à une clef déjà existante, la valeur précédente est écrasée.

Dictionnaires

Exemple :

```
>>> d = {'entier': 1, 'liste': [1, 2, 3], 3: 'trois', 'dico': eng2fr}
>>> print(d)
{'entier': 1, 'liste': [1, 2, 3], 3: 'trois', 'dico': {'two': 'deux',
'one': 'un', 'four': 'quatre', 'three': 'trois'}}
>>> print(d['entier'])
1
>>> print(d['liste'])
[1, 2, 3]
>>> print(d[3])
'trois'
>>> d[[1, 2]] = 'liste comme clef?'
TypeError: unhashable type: 'list'
>>> print(d['dico']['two'])
'deux'
```

Réutilisation de concepts déjà vus pour d'autres structures

- fonction `len`

```
>>> len(eng2fr)
4
```

- opérateur `in` : il s'applique pour rechercher une clef, pas une valeur (**intuition** : nous avons construit un dictionnaire anglais-français, pas l'inverse).

```
>>> 'one' in eng2fr
True
>>> 'deux' in eng2fr
False
```

- par contre on ne pourra pas utiliser le “slice” puisqu'on ne travaille pas avec des indices entiers.

Supprimer un élément d'un dictionnaire

La méthode `pop` existe aussi pour les dictionnaires, mais elle a besoin d'une clef (qui existe).

Elle retourne la **valeur** de l'élément supprimé.

```
>>> removed_element = eng2fr.pop('one')
>>> print(eng2fr)
{'two': 'deux', 'four': 'quatre', 'three': 'trois'}
>>> print(removed_element)
un
>>> eng2fr.pop('quatre')
KeyError: 'quatre'
```

Itérer sur un dictionnaire

Si on applique une boucle `for` sur un dictionnaire, on itère sur les **clefs** (pas les valeurs). Ce n'est pas un problème puisqu'on accède facilement à une valeur via sa clef.

```
>>> for k in eng2fr:  
    print(k, eng2fr[k])
```

```
two deux  
four quatre  
three trois
```

Itérer sur un dictionnaire

Pour itérer sur les valeurs, on peut utiliser la méthode `values` qui retourne un objet itérable `dict_values` (on appelle ce genre d'objets une “vue” car il permet de voir, ici, les valeurs).

```
>>> vals = eng2fr.values()
>>> type(vals)
<class 'dict_values'>
>>> print(vals)
dict_values(['deux', 'quatre', 'trois'])
>>> for v in vals:
    print(v)
```

```
deux
quatre
trois
```

Au besoin, on peut convertir une vue en liste.

```
>>> valeurs = list(vals)
>>> print(valeurs)
['deux', 'quatre', 'trois']
```

Itérer sur un dictionnaire trié

Si on veut obtenir les clefs d'un dictionnaire de façon triée, on peut utiliser la fonction `sorted`.

```
>>> sorted(eng2fr)
['four', 'three', 'two']
>>> for k in sorted(eng2fr):
    print(k, ': ', eng2fr[k])
```

```
four : quatre
three : trois
two : deux
```

Vues

On a vu la méthode `values` qui retourne une **vue** sur les valeurs d'un dictionnaire. Il existe aussi

- une méthode `keys` qui retourne une vue sur les clefs ;
- une méthode `items` qui retourne une vue sur les paires clef-valeur.

```
>>> mon_dict = {"nom": "Alice", "âge": 25, "ville": "Paris"}
>>> print(mon_dict.items())
dict_items([('nom', 'Alice'), ('âge', 25), ('ville', 'Paris')])
>>> for clef, valeur in mon_dict.items():
...     print(f"Clef : {clef} -> Valeur : {valeur}")
```

```
Clef : nom -> Valeur : Alice
Clef : âge -> Valeur : 25
Clef : ville -> Valeur : Paris
```

Comparaison de dictionnaires

En Python, les dictionnaires sont comparables, mais seulement avec `==` et `!=`.

Deux dictionnaires sont égaux si et seulement si ils ont les mêmes clefs et les mêmes valeurs associées.

```
>>> d1 = {"a": 1, "b": 2}
>>> d2 = {"b": 2, "a": 1}
>>> print(d1 == d2)
True
```

L'ordre n'a pas d'importance, car un dictionnaire est une structure non ordonnée. En conséquence, les opérateurs `<`, `>`, `<=` et `>=` ne sont pas définis pour les dictionnaires.

```
>>> print(d1 < d2)
TypeError: '<' not supported between instances of 'dict' and 'dict'
```

Comparaison de dictionnaires

Par contre, grâce aux vues, on peut comparer les clefs ou les valeurs de deux dictionnaires.

```
>>> d1 = {"a": 1, "b": 2}
>>> d2 = {"a": 1, "b": 3, "c": 4}
>>> print(d1.keys() < d2.keys())
True
```

Cependant, pour les valeurs, il faut d'abord convertir la vue.

```
>>> print(d1.values() < d2.values())
TypeError: '<' not supported between instances of 'dict_values' and 'dict_values'
>>> print(list(d1.values()) < list(d2.values()))
True
```

Exemple d'utilisation des dictionnaires

Problème

Comment compter la fréquence de chaque lettre dans une chaîne de caractères ?

Plusieurs solutions :

- utiliser 26 variables (compteurs), une pour chaque lettre.
- utiliser une liste de 26 compteurs.
- utiliser un dictionnaire.

Essayons ces possibilités.

Compter la fréquence des lettres

Solution 1 : créer 26 variables, une pour chaque lettre. Traverser la chaîne et, pour chaque caractère, incrémenter le compteur correspondant, par ex. en utilisant une instruction conditionnelle chaînée (26 cas!).

Démonstration du code en séance : il est effrayant.

Compter la fréquence des lettres

Solution 2 : créer une liste de 26 éléments puis convertir chaque caractère en un nombre (en utilisant la fonction `ord`). Utiliser ce nombre comme indice de la liste pour incrémenter le compteur approprié. Pour réafficher une lettre à partir d'un indice, utiliser `enumerate` et `chr`.

Démonstration du code en séance : c'est déjà mieux mais le travail sur les indices est gênant.

Compter la fréquence des lettres

Solution 3 : créer un dictionnaire avec les caractères comme clefs et les compteurs comme valeurs. La première fois que l'on rencontre un caractère, on l'ajoute dans le dictionnaire avec une valeur 1. Lors d'une prochaine occurrence de la lettre, on incrémente la valeur.

Démonstration du code en séance : code simplifié et qui n'utilise que les compteurs réellement utiles. De plus, il associe directement une lettre (la clef) à ce qu'on veut calculer sur celle-ci.

Tuples comme clefs d'un dictionnaire

Il est fréquent d'utiliser des tuples comme clefs d'un dictionnaire (principalement car on ne peut pas utiliser des listes).

Par exemple, on peut utiliser un dictionnaire pour stocker un répertoire téléphonique dont les clefs sont un tuple (nom, prénom).

```
>>> tel = {}  
>>> tel['Baroud', 'Bill'] = '065-37-07-56'  
>>> tel['II', 'Albert'] = '02-256-89-14'  
>>> tel['Poelvoorde', 'Benoit'] = '081-23-89-65'  
>>> for last, first in tel:  
    print(first, last, tel[last, first])
```

```
Benoit Poelvoorde 081-23-89-65  
Bill Baroud 065-37-07-56  
Albert II 02-256-89-14
```

JSON et dictionnaires

En réalité, vous aviez déjà rencontré un dictionnaire dans le cours d'*Algorithmique et programmation impérative*.

JSON est un format de données textuelles qui permet de stocker des données structurées. Il est très utilisé pour échanger des données entre applications. En Python, le module `json` utilise un dictionnaire.

```
>>> import json
>>> data = {"nom": "Alice", "age": 25, "ville": "Paris"}
>>> json_data = json.dumps(data)
>>> print(json_data)
{"nom": "Alice", "age": 25, "ville": "Paris"}
(...)
>>> data2 = json.loads(json_data)
>>> print(data2)
{'nom': 'Alice', 'age': 25, 'ville': 'Paris'}
>>> print(type(data2))
<class 'dict'>
```

Exercice

Vous êtes en train de créer un système de gestion des joueurs pour un jeu en ligne. Chaque joueur est identifié par un `ID`, et son pseudo et son score sont enregistrés dans un dictionnaire.

Notre mission :

- Créer un dictionnaire pour les joueurs :

```
joueurs = { 101: ("Shadow", 1200),  
            202: ("Blaze", 950),  
            303: ("Ghost", 1340) }
```
- Ajouter un nouveau joueur "Phoenix" avec l'ID 404 et un score de 1100.
- Mettre à jour le score du joueur "Blaze" à 1020.

Suite slide suivant.

Exercice

- Changer le nom du joueur "Ghost" en "Spectre".
- Supprimer "Shadow" de l'annuaire (il a quitté le jeu).
- Afficher tous les joueurs avec leur score, en affichant les meilleurs joueurs d'abord.
- Sauvegarder les scores dans un fichier.

Ensembles

Ensembles

Un **ensemble** est une structure de données dynamique et mutable.

C'est une collection **non ordonnée** d'éléments **uniques**, comme en mathématique.

Techniquement, un ensemble en Python est hétérogène, mais nous ne l'utiliserons que pour des données **homogènes**.

Ensembles

Syntaxe :

Un ensemble est délimité par des accolades { }. Les éléments sont séparés par des virgules.

La différence avec les dictionnaires : il n'y a pas de paire clef-valeur (et donc de :), uniquement des éléments. On peut voir un ensemble comme un dictionnaire qui ne contient que des clefs, sans valeurs associées.

```
>>> s = {1, 2, 3}
>>> type(s)
<class 'set'>
>>> print(s)
{1, 2, 3}
>>> d = {1: 'un', 2: 'deux'}
>>> type(d)
<class 'dict'>
```

Ensembles

Un ensemble vide est créé avec `set()` (car `{}` crée un dictionnaire vide).

```
>>> s = {}  
>>> type(s)  
<class 'dict'>  
>>> s = set()  
>>> type(s)  
<class 'set'>
```

Ensembles

Un ensemble est **non ordonné**, donc on ne peut pas accéder à un élément par un indice. Le but d'un ensemble, en terme de données, est principalement de tester l'appartenance d'un élément à cet ensemble.

```
>>> s = {1, 2, 3}
>>> s[0]
TypeError: 'set' object does not support indexing
>>> 2 in s
True
>>> 4 in s
False
```

Ensembles

On peut ajouter un élément à un ensemble avec la méthode `add`.

Un ensemble contient des données **uniques**, donc si on ajoute un élément déjà présent, il ne sera pas ajouté une seconde fois.

```
>>> s = {1, 2, 3}
>>> s.add(4)
>>> print(s)
{1, 2, 3, 4}
>>> s.add(2)
>>> print(s)
{1, 2, 3, 4}
```

Ensembles

Par contre, on ne peut pas ajouter un élément si celui-ci n'est pas hachable.

```
>>> s.add([1, 2, 3])  
TypeError: unhashable type: 'list'
```

Le fait que les éléments d'un ensemble soient hachables est une conséquence de la structure sous-jacente des ensembles en Python, et cela permet d'obtenir des performances intéressantes pour certaines opérations (voir plus loin).

Ceci renforce l'idée qu'un ensemble est une sorte de dictionnaire qui ne contient que des clefs (qui elles aussi doivent être hachables et sont uniques).

Ensembles

On peut retirer un élément d'un ensemble avec la méthode `remove`. Si l'élément n'est pas présent, une exception est levée.

```
>>> s = {1, 2, 3}
>>> s.remove(2)
>>> print(s)
{1, 3}
>>> s.remove(2)
KeyError: 2
```

Ensembles

On peut aussi utiliser la méthode `discard` pour retirer un élément. Si l'élément n'est pas présent, aucune exception n'est levée.

```
>>> s = {1, 2, 3}
>>> s.discard(2)
>>> print(s)
{1, 3}
>>> s.discard(2)
>>> print(s)
{1, 3}
```

Ensembles

La méthode `union` permet d'obtenir l'union de deux ensembles.

L'opérateur `|` permet de faire la même chose.

```
>>> s = {1, 2, 3}
>>> t = {3, 4, 5}
>>> u = s.union(t)
>>> print(u)
{1, 2, 3, 4, 5}
>>> v = {5, 6, 7}
>>> w = u | v
>>> print(w)
{1, 2, 3, 4, 5, 6, 7}
```

Ensembles

La méthode `intersection` permet d'obtenir l'intersection de deux ensembles.

L'opérateur `&` permet de faire la même chose.

```
>>> s = {1, 2, 3}
>>> t = {3, 4, 5}
>>> i = s.intersection(t)
>>> print(i)
{3}
>>> u = {2, 3, 4}
>>> v = i & u
>>> print(v)
{3}
```

Ensembles

On peut utiliser la méthode `difference` pour obtenir la différence (ensembliste) de deux ensembles.

L'opérateur `-` permet de faire la même chose.

Contrairement à l'union et l'intersection, cette opération n'est pas commutative.

```
>>> s = {1, 2, 3}
>>> t = {3, 4, 5}
>>> u = s.difference(t)
>>> print(u)
{1, 2}
>>> v = t - s
>>> print(v)
{4, 5}
```

Ensembles

Pour savoir si un ensemble est un sous-ensemble d'un autre, on peut utiliser la méthode `issubset`.

```
>>> s = {1, 2, 3}
>>> t = {3, 4, 5}
>>> u = {1, 2}
>>> print(u.issubset(s))
True
>>> print(t.issubset(s))
False
```

Comparaison d'ensembles

Même si les ensembles sont non-ordonnés, et contrairement aux dictionnaires, ils sont comparables pour tous les opérateurs.

C'est logique : les clefs d'un dictionnaire sont comparables et un ensemble est comme un dictionnaire sans valeurs. De plus, les comparaisons d'ensembles ont un sens en mathématiques, au sens de l'inclusion.

```
>>> s1 = {1, 2}
>>> s2 = {1, 2, 3}
>>> s3 = {1, 2}
>>> print(s1 < s2) # sous ensemble strict ?
True
>>> print(s1 < s3) # sous ensemble strict ?
False
>>> print(s1 <= s2) # sous ensemble ?
True
>>> print(s1 == s3) # ensemble égaux ?
True
```

Exercice

Vous travaillez dans une librairie en ligne. Vous avez plusieurs ensembles d'auteurs provenant de différents catalogues de livres. L'objectif est de faire un peu de gestion pour mieux organiser les données des auteurs.

Notre mission :

- Créer un catalogue `roman` contenant {"Victor Hugo", "J.K. Rowling", "George Orwell", "Hemingway"}.
- Créer un catalogue `science_fiction` contenant {"Isaac Asimov", "Arthur C. Clarke", "J.K. Rowling", "H.G. Wells"}.
- Créer un catalogue `poesie` contenant {"Arthur Rimbaud", "Victor Hugo", "Charles Baudelaire", "Hemingway"}.

Suite slide suivant.

Exercice

- Trouver les auteurs qui apparaissent à la fois dans les romans et la poésie.
- Trouver les auteurs qui apparaissent dans au moins l'un des catalogues (romans, science-fiction ou poésie).
- Trouver les auteurs qui sont présents uniquement dans les romans et pas dans les autres catalogues.
- Afficher tous les auteurs présents dans le catalogue de science-fiction mais pas dans les romans ou la poésie.

Ensembles immuables (Frozen sets)

Un **frozen set** est un ensemble immuable, c'est-à-dire qu'on ne peut pas lui ajouter ou lui retirer des éléments.

On peut créer un ensemble immuable avec la fonction `frozenset` (qui prend en argument un itérable).

```
>>> s = frozenset([1, 2, 3])
>>> print(s)
frozenset({1, 2, 3})
>>> s.add(4)
AttributeError: 'frozenset' object has no attribute 'add'
>>> frozenset("hello")
frozenset({'l', 'o', 'h', 'e'})
>>> s = {1, 2, 3}
>>> frozenset(s)
frozenset({1, 2, 3})
```

Ensembles immuables (Frozen sets)

Pourquoi utiliser des ensembles immuables ?

- Comme les frozen sets sont immuables, ils sont hachables.
- Ils peuvent donc être utilisés comme clefs dans un dictionnaire.
- Ils peuvent donc aussi être utilisés comme éléments d'un ensemble.

Exemple : on peut utiliser un ensemble de frozen sets pour représenter un ensemble de parties d'un ensemble donné.

```
>>> s = {1, 2, 3}
>>> p = {frozenset({1, 2}), frozenset({2, 3})}
>>> print(p)
{frozenset({1, 2}), frozenset({2, 3})}
```

Ensembles immuables (Frozen sets)

Toutes les méthodes des ensembles sont disponibles pour les frozen sets, sauf celles qui modifient l'ensemble.

```
>>> s = frozenset({1, 2, 3})
>>> print(s)
frozenset({1, 2, 3})
>>> print(s.union({3, 4}))
frozenset({1, 2, 3, 4})
>>> s.add(4)
AttributeError: 'frozenset' object has no attribute 'add'
```

Par exemple, l'union fonctionne car elle **retourne** un nouvel ensemble, mais l'ensemble d'origine reste inchangé.

Exercice

Dans une application de gestion d'accès, vous voulez organiser les permissions d'un utilisateur à l'aide des frozen sets, car les permissions doivent être immuables et ne peuvent pas être modifiées après leur attribution.

Notre mission :

- Créer un frozen set `permissions_utilisateur` avec les permissions suivantes : "lecture", "écriture", et "exécution".
- Créer un autre frozen set `permissions_administrateur` avec les permissions "lecture", "écriture", "exécution", et "supprimer".
- Créer un troisième frozen set `permissions_invité` avec la seule permission "lecture".

Suite slide suivant.

Exercice

- Vérifier si l'utilisateur a la permission "écriture".
- Trouver les permissions communes entre les utilisateurs et les administrateurs.
- Afficher les permissions de l'administrateur qui ne sont pas présentes chez l'utilisateur.
- Vérifier si les permissions d'un invité sont un sous-ensemble des permissions de l'utilisateur.

Complexités

Complexité des opérations

Nous n'allons pas détailler les complexités de toutes les opérations sur les structures de données, mais nous allons nous concentrer sur

- les opérations qui sont efficaces, et qui donc motivent l'utilisation d'une structure en particulier ;
- les opérations qui sont moins efficaces, et qui donc peuvent être un frein à l'utilisation d'une structure.

Dans ce qui suit, nous noterons n la taille de la structure de données (le nombre d'éléments dans une liste, un tuple, un ensemble, ou un dictionnaire).

Complexité des opérations sur des structures ordonnées

Accéder à un élément **via son indice** est une opération qui est efficace pour les structures ordonnées (listes et les tuples).

Le coût de cette opération est en $O(1)$.

Par contre, insérer ou supprimer un élément **à un indice donné** dans une liste est une opération qui est moins efficace.

- Dans le pire des cas, la complexité est en $O(n)$ (insérer ou supprimer au début de la liste).
- En moyenne, la complexité est en $O(n/2)$.
- Dans le meilleur des cas, la complexité est en $O(1)$ (insérer ou supprimer à la fin de la liste, comme le font `append` et `pop`).

La raison vient du fait que pour rendre l'accès en $O(1)$, Python doit décaler les éléments de la liste.

Complexité des opérations sur des structures ordonnées

Rechercher une valeur dans un tuple ou une liste (non triés) est une opération qui est en $O(n)$ dans le pire des cas (par exemple : opérateur `in`).

Il en va donc de même pour calculer le **maximum** ou le **minimum** dans une liste ou un tuple (par exemple, utiliser les fonctions `max` et `min`).

Complexité des opérations sur des structures ordonnées

Par contre, obtenir le **nombre d'éléments** via `len` dans une liste ou un tuple est une opération qui est en $O(1)$ (le nombre d'éléments est conservé par Python et mis à jour lors des modifications).

Le **tri** utilisé en Python (via la méthode `sort` ou la fonction `sorted`) est en $O(n \log n)$. Python utilise l'algorithme **Timsort** qui est un algorithme hybride dérivé du tri par fusion et du tri par insertion.

Complexité des opérations sur les dictionnaires

Les dictionnaires ont une structure (sous-jacente) en Python qui leur permet des performances très intéressantes pour accéder à un élément **en moyenne** via sa **clef unique** (mais pas toujours dans le pire des cas). Ils utilisent ce qu'on appelle une table de hachage².

En pratique, la complexité en moyenne sera très souvent atteinte.

	Moyenne	Pire des cas
Accès à un élément (via la clef)	$O(1)$	$O(n)$
Mettre à jour un élément (via la clef)	$O(1)$	$O(n)$
Insérer un élément (via la clef)	$O(1)$	$O(n)$
Supprimer un élément (via la clef)	$O(1)$	$O(n)$

2. Voir cours *Structure des données II*.

Complexité des opérations sur les dictionnaires

En Python, obtenir une vue d'un dictionnaire (par exemple, avec `keys()`, `values()`, ou `items()`) est une opération qui est en $O(1)$.

Par contre, ces vues sont souvent utilisées pour itérer sur les éléments du dictionnaire via par exemple une boucle `for`, et dans ce cas, la complexité est en $O(n)$ (multiplié par la complexité de l'opération à réaliser pour chaque élément).

Complexité des opérations sur les dictionnaires

Pour tester cette complexité en moyenne, nous allons réaliser le test suivant :

- lire le fichier `words.txt` pour créer une liste t et un dictionnaire d contenant les 113809 mots ;
- utiliser le module `umons_cpu` pour évaluer le temps d'exécution de la recherche d'un mot :
 - ▶ avec une recherche linéaire dans la liste (complexités moyenne et pire des cas en $O(n)$) ;
 - ▶ avec une recherche dichotomique dans la liste, puisque les mots sont triés (complexités en moyenne et pire des cas en $O(\log_2 n)$) ;
 - ▶ avec accès direct dans un dictionnaire (complexité en moyenne en $O(1)$, dans le pire des cas en $O(n)$).

Complexité des opérations sur les dictionnaires

Création des données :

```
def get_data_structures():  
    t = []  
    d = {}  
    with open('words.txt') as file:  
        for line in file:  
            word = line.strip()  
            t.append(word)  
            d[word] = ''  
    return t, d
```

Complexité des opérations sur les dictionnaires

Recherche linéaire :

```
def linear_search(t, w):  
    return w in t
```

Recherche dichotomique :

```
def dichotomic_search(t, w):  
    start = 0  
    end = len(t) - 1  
    mid = (end - start) // 2  
    while (end - start > 1) and w != t[mid]:  
        if w < t[mid]:  
            end = mid - 1  
        else:  
            start = mid + 1  
        mid = start + (end - start) // 2  
    return len(t) > 0 and w == t[mid]
```

Complexité des opérations sur les dictionnaires

Recherche dans le dictionnaire :

```
def dico_search(d, w):  
    return w in d
```

Pour obtenir un mot aléatoire :

```
import random  
  
def random_word(t):  
    index = random.randint(0, len(t)-1)  
    return t[index]
```

Complexité des opérations sur les dictionnaires

Résultats obtenus en faisant appel au module `umons_cpu` : on fait deux types de tests (mots existants ou non).

```
Temps affichés en msec
Tests sur mots aleatoires (existants):
linear      dicho      dict
0.055249 0.002190 0.000055
Tests sur mots non presents:
linear      dicho      dict
1.160452 0.002120 0.000061
```

Remarque : dans les tests sur les mots aléatoires, le coût de la création d'un mot aléatoire est inclus dans les temps ci-dessus, mais il est le même pour les 3 méthodes.

Complexité des opérations sur les ensembles

Les ensembles utilisent également des tables de hachages.

	Moyenne	Pire des cas
Insertion d'un élément	$O(1)$	$O(n)$
Suppression d'un élément	$O(1)$	$O(n)$
Vérification d'appartenance	$O(1)$	$O(n)$
Parcours des éléments	$O(n)$	$O(n)$
Union	$O(n + m)$	$O(n + m)$
Intersection	$O(\min(n, m))$	$O(n)$
Différence	$O(n)$	$O(n)$

Complexité des opérations sur les ensembles

En *Algorithmique et programmation impérative*, nous avons résolu le problème suivant en utilisant une liste.

Exercice

Ecrire un programme qui affiche les paires de mots (contenus dans le fichier `words.txt`) qui sont l'inverse l'un de l'autre. Par exemple, `stop` et `pots`.

Son efficacité n'était pas terrible !

Démonstration en séance : version améliorée (plus rapide et sans doublons) basée sur un set.

Quand utiliser quelle structure de données ?

Quand utiliser quelle structure de données ?

- **Chaînes** : pour stocker des textes.
- **Tuples** : pour stocker un nombre fixés de données.
- **Listes** : pour stocker des données modifiables et ordonnées.
- **Dictionnaires** : pour stocker des données avec une clef unique, et que vous avez besoin d'un accès rapide via la clef.
- **Ensembles** : pour stocker des données uniques et effectuer des opérations ensemblistes.
- **Frozen sets** : pour stocker des ensembles immuables, souvent comme clef de dictionnaire ou dans des structures de données immuables (comme les tuples).

Quelle séquence choisir ? Exercice

- 1 Vous devez stocker un ensemble de **noms d'utilisateurs** uniques et vous voulez effectuer des opérations rapides de recherche pour savoir si un utilisateur existe déjà.
- 2 Vous devez faire un **panier de shopping** pour votre e-commerce.
- 3 Vous voulez stocker les **coordonnées GPS** de votre domicile et de l'UMONS.
- 4 Vous devez stocker des informations sur des **étudiants** en les identifiant via leur matricule.
- 5 Vous voulez représenter un **nom complet** d'une personne (prénom, nom de famille), mais vous savez qu'une fois créé, il ne changera pas.
- 6 Vous devez **vérifier rapidement si un élément** (comme une **adresse email**) existe dans une liste de contacts.

Quelle séquence choisir ? Exercice

Situation	SD	Justification
1. Utilisateurs uniques	Set	Unicité et recherche rapide
2. Panier de shopping	Liste	Ordre important, modifiable
3. Coordonnées GPS	Tuple	Immuable et ordonné
4. Etudiants	Dict	Accès rapide par clé
5. Nom complet	Tuple	Immuable et ordonné
6. Vérification d'email	Set	Recherche rapide sans doublons

Remarques et opérations fréquentes

Fonctions avec un nombre variable d'arguments

Une fonction peut prendre un nombre variable d'arguments. Par exemple les fonctions `min` et `max` peuvent être appliquées sur des séquences, mais également sur un nombre indéfini d'arguments.

```
>>> t = [5, 7, 8, 1, 3]
>>> max(t)
8
>>> max(4, 7, 3)
7
>>> min(4, 7, 3, 2)
2
```

Un autre exemple est la fonction `print` dont le nombre d'arguments est libre.

Nous allons pouvoir faire la même chose pour nos fonctions, grâce aux tuples.

Fonctions avec un nombre variable d'arguments

Pour définir ses propres fonctions avec un nombre indéfini d'arguments, on utilise un paramètre dont le nom commence par *. Ce paramètre rassemble (**gather**) les arguments en un tuple, quelque soit leur nombre.

```
>>> def f(*args):  
    print(type(args), 'of length', len(args))  
    print(args)
```

```
>>> f(2, 4)  
<class 'tuple'> of length 2  
(2, 4)  
>>> f('hello', 2.0, [1, 2])  
<class 'tuple'> of length 3  
('hello', 2.0, [1, 2])
```

Remarque : il ne peut y avoir qu'un seul paramètre de ce type, et il doit se trouver à la fin de la liste des paramètres.

Fonctions avec un nombre variable d'arguments

On peut combiner l'opérateur gather `*` avec des paramètres requis et optionnels (avec valeurs par défaut).

On commence par les paramètres requis, puis les optionnels et enfin le paramètre de taille variable.

```
def g(required, optional=0, *args):  
    print('required:', required)  
    print('optional:', optional)  
    print('others:', args)
```

Fonctions avec un nombre variable d'arguments

```
>>> g()  
TypeError: g() takes at least 1 argument (0 given)  
>>> g(1)  
required: 1  
optional: 0  
others: ()  
>>> g(1,2)  
required: 1  
optional: 2  
others: ()  
>>> g(1,2,3)  
required: 1  
optional: 2  
others: (3,)  
>>> g(1,2,3,4)  
required: 1  
optional: 2  
others: (3, 4)
```

Convertir une structure en une autre

Les fonctions `tuple`, `list`, `dict` et `set` permettent de convertir une structure en une autre, quand cela a du sens.

```
>>> t = (1, 2, 3, 2)
>>> list(t)
[1, 2, 3, 2]
>>> set(t)
{1, 2, 3}
>>> d = {'a': 1, 'b': 2}
>>> tuple(d)
('a', 'b')
>>> list(d.values())
[1, 2]
```

Structures imbriquées

On le savait déjà pour les listes, on peut imbriquer les structures entre elles.

Par exemple, on peut avoir une liste de listes, une liste de dictionnaires, un dictionnaire de listes, un tuple de listes³, etc.

Par contre, certaines choses doivent être respectées, par exemple, on ne peut pas avoir un dictionnaire comme clef d'un autre dictionnaire, car les clefs doivent être immuables.

3. Dans ce cas on ne peut pas ajouter ou supprimer une des listes, mais on peut modifier le contenu d'une des listes existantes.

Structures imbriquées

```
>>> d = {'a': [1, 2], 'b': [3, 4]}
>>> s = {(1, 2), (3, 4)}
>>> t = ([1, 2], [3, 4])
>>> l = [d, s, t]
>>> t[0][0] = 5
>>> print(l)
[{'a': [1, 2], 'b': [3, 4]}, {(1, 2), (3, 4)}, ([5, 2], [3, 4])]
>>> d2 = {}
>>> d2[d] = 1
TypeError: unhashable type: 'dict'
```

Précautions avec les structures mutables

Comme pour les listes, les autres structure mutables (dictionnaires et sets) doivent être traitées avec précaution :

- faire une (deep) copie si nécessaire ;
- faire attention aux effets de bord.

Réduction

Pour additionner tous les nombres d'une liste, on peut utiliser une boucle :

```
def add_all(t):  
    total = 0  
    for x in t:  
        total += x  
    return total  
  
>>> t = list(range(1,11))  
>>> add_all(t)  
55
```

Réduction

Additionner les éléments d'une liste (ou d'un autre itérable) est une opération fréquente. Python offre une fonction qui permet également de le faire.

```
>>> t = [1, 2, 3]
>>> sum(t)
6
```

Une opération qui combine une séquence d'éléments en une seule valeur est appelée une **réduction**.

Réduction : séquence $\rightarrow$ valeur.

Mapping

Autre opération fréquente : traverser une séquence pour en construire une nouvelle. Par exemple, la fonction suivante prend une liste de chaînes en argument et retourne une liste qui “capitalise” les mots de la liste de départ⁴.

```
def capitalize_all(t):
    res = []
    for s in t:
        res.append(s.capitalize())
    return res

>>> liste = ['hello', 'bonjour', 'dag']
>>> capitalize_all(liste)
['Hello', 'Bonjour', 'Dag']
```

Une opération qui applique une fonction (ici, `capitalize`) sur chaque élément d'une séquence est appelée un **mapping**.

Mapping : séquence $\rightarrow$ séquence.

4. On utilise la méthode `capitalize` sur chaque chaîne : elle met la première lettre en majuscule.

Filtre

Autre opération fréquente : filtrer une séquence. Par exemple, la fonction suivante ne retourne que les chaînes qui sont en majuscules.

```
def only_upper(t):  
    res = []  
    for s in t:  
        if s.isupper():  
            res.append(s)  
    return res  
  
>>> liste = ['Hello', 'bonjour', 'DAG']  
>>> only_upper(liste)  
['DAG']
```

- une opération qui sélectionne certains éléments pour retourner une sous-séquence est appelée un **filtre**.

Filtre : séquence $\rightarrow$ sous-séquence.

Exercice

Exercice

Ecrire une fonction qui prend une liste de nombres entiers en paramètres et qui retourne la somme cumulative, c-à-d une nouvelle liste telle que l'élément d'indice i soit la somme des $i + 1$ premiers éléments.

Par exemple, la somme cumulative de $[1, 2, 3]$ est $[1, 3, 6]$.

Exercice

```
def cumul_sum(t):  
    res = []  
    s = 0  
    for x in t:  
        s += x  
        res.append(s)  
    return res  
  
>>> cumul_sum([1,2,3])  
[1, 3, 6]  
>>> cumul_sum(list(range(10)))  
[0, 1, 3, 6, 10, 15, 21, 28, 36, 45]
```

Fonctions Python pour les opérations fréquentes

La plupart des opérations sur des séquences peuvent être exprimées comme une combinaison de mappings, de filtres et de réductions.

Python possède des fonctions pour réaliser directement deux de ces opérations : `map`, `filter`. Une réduction est en général réalisée via l'utilisation d'un accumulateur et d'une boucle `for`.

Fonction map

`map(function, *iterables) -> map`

Applique une action (fonction ou méthode) sur chaque élément d'un objet itérable et retourne un objet `map` qui est itérable ou peut être converti en liste.

```
>>> import math
>>> list(map(math.sqrt, [2, 4, 6, 100]))
[1.4142135623730951, 2.0, 2.449489742783178, 10.0]
>>> list(map(str.upper, 'hello'))
['H', 'E', 'L', 'L', 'O']
>>> def double(lettre):
    return lettre * 2

>>> for l in map(double, 'Hello'):
    print(l, end = ', ')

HH, ee, ll, ll, oo,
```

Fonction `map`

Dans le code précédent, on a défini une fonction `double` pour pouvoir utiliser `map` :

Mais cette fonction est une “mini” fonction, qui pourrait donc être remplacée par une fonction `lambda`, pour faire le même travail en une seule ligne.

```
>>> print(list(map(lambda x: x * 2, 'Hello')))  
['HH', 'ee', 'll', 'll', 'oo']
```

Fonction `filter`

`filter(function or None, iterable) → filter object`

Sélectionne les éléments qui satisfont un certain critère (fonction ou méthode booléenne) et les retourne sous forme d'un objet `filter` qui est itérable ou peut être converti en liste.

```
>>> list(filter(str.isalpha, 'r2d2'))  
['r', 'd']  
>>> list(filter(lambda x : x % 2 == 0, [1, 2, 4, 5, 7, 10, 11]))  
[2, 4, 10]
```

Remarque : si la fonction est “None”, retourne les éléments qui sont True.

Exercice

Exercice

Ecrire une fonction qui, à partir d'une liste d'entiers en entrée, retourne une liste contenant les carrés des nombres de cette liste, mais uniquement si ceux-ci se terminent par le chiffre 6.

Challenge : le corps de votre fonction peut être écrit en une seule ligne !

Exercice

Version longue

```
def square(x):  
    return x ** 2  
  
def six_ended(x):  
    return x % 10 == 6  
  
def six_ended_squares(t):  
    squares = list(map(square, t))  
    res = list(filter(six_ended, squares))  
    return res  
  
print(six_ended_squares(list(range(100))))
```

Exercice

Version courte

```
def six_ended_squares(t):  
    return list(filter(lambda x: x % 10 == 6, list(map(lambda x: x ** 2, t))))  
  
print(six_ended_squares(list(range(100))))
```

Compréhension

Le concept de **compréhension** permet de créer une structure itérable à partir d'une autre, en une seule ligne.

Syntaxe (liste) : `[expression for element in iterable]`

Exemple : Créer une liste des carrés des nombres de 0 à 4 :

```
>>> squares = [x**2 for x in range(5)]  
>>> print(squares)  
[0, 1, 4, 9, 16]
```

Il est possible d'ajouter une condition pour filtrer les éléments :

```
>>> evens = [x for x in range(10) if x % 2 == 0]  
>>> print(evens)  
[0, 2, 4, 6, 8]
```

Compréhension

La syntaxe est similaire pour les autres structures :

```
>>> tuple(x**2 for x in range(5)) # tuple
(0, 1, 4, 9, 16)
>>> {x: x**2 for x in range(5)} # dict
{0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
>>> {x**2 for x in range(5)} # set
{0, 1, 4, 9, 16}
```

Compréhension

Avec un peu d'imagination, on peut faire beaucoup de choses avec les compréhensions.

Exemples :

```
>>> words = ["chouette", "le", "monde", "de", "python"]
>>> upper_words = [w.upper() for w in words if len(w) > 4]
>>> print(upper_words)
['CHOUETTE', 'MONDE', 'PYTHON']
```

```
>>> names = ["Alice", "Bob", "Charlie"]
>>> name_lengths = {name: len(name) for name in names}
>>> print(name_lengths)
{'Alice': 5, 'Bob': 3, 'Charlie': 7}
```

```
>>> ages = {"Alice": 15, "Bob": 30, "Charlie": 22}
>>> adultes = {k: v for k, v in ages.items() if v >= 18}
>>> print(adultes)
{'Bob': 30, 'Charlie': 22}
```

Compréhension

Exemples :

```
>>> import random
>>> alea = [random.randint(0,10) for _ in range(6)]
>>> print(alea)
[9, 5, 10, 8, 2, 2]
>>> uniques = {x for x in alea}
>>> print(uniques)
{2, 5, 8, 9, 10}

>>> mot = "universite"
>>> voyelles = {c for c in mot if c in "aeiou"}
>>> print(voyelles)
{'i', 'e', 'u'}
```

Compréhension

Exemples :

```
>>> parity = ["pair" if x % 2 == 0 else "impair" for x in range(5)]
>>> print(parity)
['pair', 'impair', 'pair', 'impair', 'pair']

>>> matrix = [[x for x in range(3)] for _ in range(3)]
>>> print(matrix)
[[0, 1, 2], [0, 1, 2], [0, 1, 2]]
```

C'est donc puissant, mais il ne faut pas en abuser : la lisibilité du code est importante.